MeepleAI - API Reference

Documentazione API REST, endpoint, health check, permessi, rate limiting, session tracking.

Data generazione: 8 marzo 2026

File inclusi: 9

Indice

1. api/README.md
2. api/bgg-rate-limiting.md
3. api/bounded-contexts/game-management/play-records.md
4. api/bounded-contexts/game-management/play-sessions.md
5. api/endpoints/library-game-detail.md
6. api/health-check-api.md
7. api/multi-agent-system.md
8. api/permission-api-reference.md
9. api/session-tracking/sse-integration.md

---

api/README.md

API Documentation

MeepleAI REST API - Complete reference per integrazione backend

Quick Access

Resource	URL
Scalar UI (Interactive)	http://localhost:8080/scalar/v1
OpenAPI Spec	http://localhost:8080/openapi/v1.json
Health Check	http://localhost:8080/health
Metrics	http://localhost:8080/metrics

Authentication

MeepleAI supporta **3 metodi di autenticazione**:

1. Cookie-Based Authentication

Utilizzato da: Frontend web application

Flow: *(blocco di codice rimosso)*

Example: *(blocco di codice rimosso)*

2. API Key Authentication

Utilizzato da: External integrations, scripts, mobile apps

Format: `mpl_{env}_{base64}` (PBKDF2 hashed in DB)

Example: *(blocco di codice rimosso)*

Generate API Key: *(blocco di codice rimosso)*

3. OAuth 2.0

Providers: Google, Discord, GitHub

Flow: *(blocco di codice rimosso)*

Example: *(blocco di codice rimosso)*

API Endpoints Overview

Authentication (`/api/v1/auth`)

Method	Endpoint	Description	Auth
POST	`/register`	Create new user account	None
POST	`/login`	Login with email/password	None
POST	`/logout`	Invalidate session	Cookie
GET	`/me`	Get current user profile	Cookie/API Key
POST	`/2fa/enable`	Enable TOTP 2FA	Cookie
POST	`/2fa/verify`	Verify TOTP code	Cookie
POST	`/api-keys`	Generate API key	Cookie
DELETE	`/api-keys/{id}`	Revoke API key	Cookie
GET	`/oauth/{provider}`	Start OAuth flow	None
GET	`/oauth/callback/{provider}`	OAuth callback	None

Games (`/api/v1/games`)

Method	Endpoint	Description	Auth
GET	`/`	List all games (paginated)	None
GET	`/{id}`	Get game by ID	None
POST	`/`	Create new game	Admin
PUT	`/{id}`	Update game	Admin
DELETE	`/{id}`	Delete game	Admin
GET	`/{id}/rules`	Get game rules (PDF links)	None

Knowledge Base (`/api/v1/chat`)

Method	Endpoint	Description	Auth
POST	`/`	Ask question (SSE streaming)	Cookie/API Key
GET	`/threads`	List chat threads	Cookie/API Key
GET	`/threads/{id}`	Get thread by ID	Cookie/API Key
DELETE	`/threads/{id}`	Delete thread	Cookie/API Key

Documents (`/api/v1/documents`)

Method	Endpoint	Description	Auth
POST	`/upload`	Upload PDF rulebook	Cookie/API Key
GET	`/`	List uploaded documents	Cookie/API Key
GET	`/{id}`	Get document metadata	Cookie/API Key
DELETE	`/{id}`	Delete document	Cookie/API Key
GET	`/{id}/status`	Get processing status	Cookie/API Key

Administration (`/api/v1/admin`)

Method	Endpoint	Description	Auth
GET	`/users`	List all users	Admin
GET	`/stats`	System statistics	Admin
GET	`/alerts`	Active alerts	Admin
POST	`/configuration`	Update config	Admin
GET	`/configuration`	Get all config keys	Admin

Request/Response Examples

Register User

Request: *(blocco di codice rimosso)*

Response (201 Created): *(blocco di codice rimosso)*

Login

Request: *(blocco di codice rimosso)*

Response (200 OK): *(blocco di codice rimosso)*

Ask Question (SSE Streaming)

Request: *(blocco di codice rimosso)*

Response (200 OK - SSE Stream): *(blocco di codice rimosso)*

Upload PDF

Request: *(blocco di codice rimosso)*

Response (202 Accepted): *(blocco di codice rimosso)*

Get Processing Status

Request: *(blocco di codice rimosso)*

Response (200 OK): *(blocco di codice rimosso)*

Error Responses

Standard Error Format

Tutte le risposte di errore seguono RFC 7807 (Problem Details):

(blocco di codice rimosso)

Common HTTP Status Codes

Code	Meaning	Example
200	OK	Successful GET/PUT
201	Created	Successful POST (resource created)
202	Accepted	Async operation started
204	No Content	Successful DELETE
400	Bad Request	Validation errors
401	Unauthorized	Missing/invalid authentication
403	Forbidden	Insufficient permissions
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate resource (e.g., email)
422	Unprocessable Entity	Business logic validation failed
500	Internal Server Error	Server-side error
503	Service Unavailable	Dependency unavailable (DB, Redis, Qdrant)

Rate Limiting

Strategy: Token bucket algorithm

Limits:

- **Anonymous**: 100 req/min per IP
- **Authenticated**: 1000 req/min per user
- **Admin**: Unlimited

Headers: *(blocco di codice rimosso)*

Rate Limit Exceeded (429): *(blocco di codice rimosso)*

Pagination

Standard Pattern: Cursor-based pagination

Request: *(blocco di codice rimosso)*

Response: *(blocco di codice rimosso)*

Webhooks

Eventi supportati:

- `document.processing.completed`
- `document.processing.failed`
- `user.created`
- `alert.triggered`

Configurazione: *(blocco di codice rimosso)*

Payload Example: *(blocco di codice rimosso)*

Signature Verification: *(blocco di codice rimosso)*

Data Models

User

(blocco di codice rimosso)

Game

(blocco di codice rimosso)

ChatMessage

(blocco di codice rimosso)

Document

(blocco di codice rimosso)

Performance

Response Times (P95)

Endpoint	Target	Actual
`/api/v1/games`	<100ms	45ms
`/api/v1/chat` (first token)	<500ms	320ms
`/api/v1/chat` (full response)	<3s	1.8s
`/api/v1/documents/upload`	<200ms	150ms

Caching Strategy

HybridCache (L1: Memory + L2: Redis):

- Games list: 5 min
- Game details: 10 min
- User profile: 1 min
- RAG results: 30 min (cache key = hash(question + gameId))

Cache Headers: *(blocco di codice rimosso)*

Security

HTTPS Only (Production)

(blocco di codice rimosso)

CORS Configuration

Allowed Origins (Production):

- `https://meepleai.com`

- `https://app.meepleai.com`

Headers: *(blocco di codice rimosso)*

Content Security Policy

(blocco di codice rimosso)

API Key Security

- **Format**: `mpl_{env}_{base64}` (32 bytes random)
- **Storage**: PBKDF2 hashed with 10,000 iterations
- **Rotation**: Recommended every 90 days
- **Revocation**: Immediate via DELETE endpoint

Client Libraries

TypeScript/JavaScript

Auto-generated client da OpenAPI spec:

(blocco di codice rimosso)

cURL Examples

Get Games: *(blocco di codice rimosso)*

Ask Question with API Key: *(blocco di codice rimosso)*

Upload PDF with Cookie: *(blocco di codice rimosso)*

Monitoring

Health Check

Endpoint: `GET /health`

Response: *(blocco di codice rimosso)*

Metrics (Prometheus)

Endpoint: `GET /metrics`

Key Metrics:

- `http_requests_total{endpoint, method, status}`
- `http_request_duration_seconds{endpoint, method}`
- `rag_query_confidence{game}`
- `pdf_processing_duration_seconds{stage}`

Versioning

Strategy: URI versioning (`/api/v1/...`)

Deprecation Policy:

- 6 months notice for breaking changes
- Old version supported for 12 months after deprecation notice
- Sunset header: `Sunset: Sat, 01 Jan 2027 00:00:00 GMT`

Resources

- Interactive API Explorer (Scalar)
- OpenAPI Specification
- Authentication Guide
- Rate Limiting Details

Version: 1.0 **Last Updated**: 2026-01-01 **API Version**: v1 **Maintainers**: Engineering Team

---

api/bgg-rate-limiting.md

BGG API Rate Limiting

Issue: #4275 - BGG Rate Limiting with User Quota Tracking **Implementation**: Tier-based request limits to prevent API abuse

---

Overview

BGG (BoardGameGeek) API endpoints are rate-limited per user tier to:

1. Prevent individual users from exhausting shared IP quota
2. Distribute BGG API capacity fairly across user base
3. Provide clear feedback on remaining quota

Rate Limits by Tier

Tier	Requests/Minute	Use Case
Free	5	Casual users, basic searches
Normal	10	Regular users, moderate activity
Premium	20	Power users, frequent searches
Editor	15	Content editors, moderate bulk operations
Admin	Unlimited	Administrators (bypassed)

Window: 60 seconds (sliding window) **Algorithm**: Token bucket via Redis

---

Affected Endpoints

All endpoints under `/api/v1/bgg/*` are rate-limited:

- `GET /api/v1/bgg/search?query={term}` - Search BGG catalog
- `GET /api/v1/bgg/games/{bggId}` - Get game details
- `GET /api/v1/bgg/thumbnails` - Batch thumbnail fetch

Note: Admin-only endpoints (`/api/v1/admin/bgg-queue/*`) are NOT rate-limited.

Response Headers

Success Response (200 OK)

(blocco di codice rimosso)

Headers:

- `X-RateLimit-Limit` : Maximum requests allowed in window
- `X-RateLimit-Remaining` : Requests remaining in current window
- `X-RateLimit-Reset` : Unix timestamp when limit resets

Rate Limited Response (429 Too Many Requests)

(blocco di codice rimosso)

Headers:

- `Retry-After` : Seconds to wait before retrying
- Other headers same as success response

Client Implementation

Checking Rate Limit Status

Before Request: *(blocco di codice rimosso)*

Handling 429 Responses

Automatic Retry (with exponential backoff): *(blocco di codice rimosso)*

UI Feedback

Rate Limit Indicator (recommended): *(blocco di codice rimosso)*

Configuration

appsettings.json

(blocco di codice rimosso)

Environment-Specific Overrides

appsettings.Testing.json (disable for CI): *(blocco di codice rimosso)*

appsettings.Production.json (stricter limits): *(blocco di codice rimosso)*

Monitoring

Metrics (Prometheus)

Counters:

- `bgg_api_requests_total{tier="Normal",status="success"}` - Total requests by tier
- `bgg_api_rate_limited_total{tier="Free"}` - 429 responses by tier

Gauges:

- `bgg_api_quota_remaining{user_id="..."}` - Current quota for active users

Histograms:

- `bgg_api_request_duration_seconds` - Request latency

Grafana Dashboard

Panels:

1. **BGG API Usage by Tier** (line chart)
2. **Rate Limit Violations** (bar chart by tier)
3. **Top Users by BGG Requests** (table)
4. **Quota Exhaustion Events** (timeline)

Alerts

Rate Limit Abuse: *(blocco di codice rimosso)*

BGG API Overuse: *(blocco di codice rimosso)*

Resilience

Fail-Open Strategy

Redis Unavailable:

- Request is ALLOWED (fail-open)
- Error logged with severity WARNING
- Header added: `X-RateLimit-Status: Error`
- Prevents Redis outage from blocking BGG access

Error Handling

Middleware Exception: *(blocco di codice rimosso)*

Testing

Manual Testing

Test Normal Tier (10 req/min): *(blocco di codice rimosso)*

Unit Tests

Coverage: `apps/api/tests/Api.Tests/Middleware/BggRateLimitMiddlewareTests.cs`

- 12 test cases covering all tiers, bypass, errors, headers

Integration Tests

Coverage: `apps/api/tests/Api.Tests/Integration/BggRateLimitIntegrationTests.cs`

- 7 test cases with real Redis and Testcontainers

Best Practices

For Developers

✅ **DO**:

- Cache BGG responses (24h) to reduce API calls
- Show quota in UI before user hits limit
- Implement retry logic with Retry-After header
- Log rate limit events for monitoring

❌ **AVOID**:

- Polling BGG API without caching
- Ignoring 429 responses
- Making BGG requests without authentication
- Bypassing rate limits in client code

For Users

Optimize BGG Usage:

- Search in personal library first (autocomplete)
- Cache favorite games locally
- Upgrade to Premium tier for higher limits
- Use BGG sparingly during peak hours

Troubleshooting

Issue: "Rate limit exceeded" but user hasn't made requests

Causes:

- Redis key not expiring properly
- Clock skew in distributed environment
- Multiple sessions for same user

Solutions:

1. Check Redis TTL: `TTL bgg:ratelimit:{userId}`
2. Verify Redis time vs server time

3. Clear Redis key manually if needed

Issue: Admin still getting rate limited

Causes:

- AdminBypass set to false in configuration
- User role not parsed correctly
- Middleware order issue (must be after authentication)

Solutions:

1. Check `BggRateLimit:AdminBypass` in appsettings.json
2. Verify user has "Admin" role in token claims
3. Ensure middleware is registered after `UseAuthentication()`

Issue: All requests fail with 429

Causes:

- Redis counter stuck at high value
- Configuration limits set too low
- System time incorrect

Solutions:

1. Flush Redis keys: `KEYS bgg:ratelimit:*` → `DEL`
2. Check configuration values in appsettings
3. Verify server time is accurate

Related

- **Issue #4275**: BGG Rate Limiting (backend)
- **Issue #4274**: BGG Search Dialog (frontend UI)
- **Issue #4276**: E2E Tests (testing)
- **Epic #3887**: Play Records
- **Redis**: Infrastructure service (docker-compose.yml)

Last Updated: 2026-02-13 **Implemented By**: BggRateLimitMiddleware.cs **Test Coverage**: ≥90% (unit + integration)

api/bounded-contexts/game-management/play-records.md

Play Records API Documentation

Overview

The Play Records feature enables users to record and track their board game play history with flexible player management, multi-dimensional scoring, and comprehensive statistics.

Bounded Context: GameManagement **Parent Epic**: #3887 **Implementation Status**: Planned

Note: This is distinct from `GameSession` (real-time AI agent play state). `PlayRecord` is for historical play logging.

Domain Model

PlayRecord (Aggregate Root)

The core entity representing a single game play session.

Key Properties

Property	Type	Description
`Id`	`Guid`	Unique session identifier
`GameId`	`Guid?`	Optional reference to Game in catalog
`GameName`	`string`	Game name (from catalog or user input)
`CreatedByUserId`	`Guid`	Session creator
`Visibility`	`PlayRecordVisibility`	Private or Group visibility
`GroupId`	`Guid?`	Optional group for shared sessions
`SessionDate`	`DateTime`	Date of play session
`StartTime`	`DateTime?`	Optional session start time
`EndTime`	`DateTime?`	Optional session end time
`Duration`	`TimeSpan?`	Calculated or manual duration
`Status`	`PlayRecordStatus`	Planned, InProgress, Completed, Archived
`Notes`	`string?`	Optional session notes
`Location`	`string?`	Optional play location
`Players`	`List<SessionPlayer>`	Participants in session
`ScoringConfig`	`SessionScoringConfig`	Scoring configuration

Factory Methods

(blocco di codice rimosso)

Behaviors

(blocco di codice rimosso)

SessionPlayer (Entity)

Represents a participant in a play session (registered user or guest).

Property	Type	Description
`Id`	`Guid`	Player identifier within session
`UserId`	`Guid?`	Optional User reference (null for guests)
`DisplayName`	`string`	Player display name
`Scores`	`List<SessionScore>`	Multi-dimensional scores

SessionScore (Value Object)

Multi-dimensional score representation.

Property	Type	Description
`Dimension`	`string`	Score dimension (e.g., "points", "ranking", "wins")
`Value`	`int`	Score value
`Unit`	`string?`	Optional display unit (e.g., "pts", "1st", "W")

Factory Methods: *(blocco di codice rimosso)*

SessionScoringConfig (Value Object)

Configuration for scoring dimensions available in the session.

Property	Type	Description
`EnabledDimensions`	`List<string>`	Active scoring dimensions
`DimensionUnits`	`Dictionary<string, string>`	Display units per dimension

Factory Methods: *(blocco di codice rimosso)*

Enumerations

PlayRecordStatus

- `Planned` - Session scheduled but not started
- `InProgress` - Session currently active

- `Completed` - Session finished
- `Archived` - Completed session archived

PlayRecordVisibility

- `Private` - Only creator can view
- `Group` - Visible to group members

Database Schema

PlayRecords Table

(blocco di codice rimosso)

Indexes: *(blocco di codice rimosso)*

SessionPlayers Table

(blocco di codice rimosso)

SessionScores Table

(blocco di codice rimosso)

CQRS Operations

Commands

CreatePlayRecordCommand

Creates a new play session.

Request: *(blocco di codice rimosso)*

Response: `Guid` (Session ID)

Endpoint: `POST /api/v1/game-management/play-sessions`

AddPlayerToSessionCommand

Adds a player to an existing session.

Request: *(blocco di codice rimosso)*

Endpoint: `POST /api/v1/game-management/play-sessions/{sessionId}/players`

RecordSessionScoreCommand

Records a score for a player in a session.

Request: *(blocco di codice rimosso)*

Endpoint: `POST /api/v1/game-management/play-sessions/{sessionId}/scores`

StartPlayRecordCommand

Marks a session as in-progress.

Endpoint: `POST /api/v1/game-management/play-sessions/{sessionId}/start`

CompletePlayRecordCommand

Completes a session and calculates duration.

Request: *(blocco di codice rimosso)*

Endpoint: `POST /api/v1/game-management/play-sessions/{sessionId}/complete`

UpdatePlayRecordCommand

Updates session details (allowed post-completion).

Request: *(blocco di codice rimosso)*

Endpoint: `PUT /api/v1/game-management/play-sessions/{sessionId}`

Queries

GetPlayRecordQuery

Retrieves full session details including players and scores.

Request: `GET /api/v1/game-management/play-sessions/{sessionId}`

Response: *(blocco di codice rimosso)*

GetUserPlayHistoryQuery

Retrieves paginated play history for a user.

Request: `GET /api/v1/game-management/play-sessions/history?userId={userId}&page=1&pageSize=20&gameId={gameId}`

Query Parameters:

- `userId` (required): User ID
- `page` (optional, default: 1): Page number
- `pageSize` (optional, default: 20): Results per page
- `gameId` (optional): Filter by specific game

Response: *(blocco di codice rimosso)*

GetPlayerStatisticsQuery

Retrieves cross-game statistics for a player.

Request: `GET /api/v1/game-management/play-sessions/statistics?userId={userId}&startDate={startDate}&endDate={endDate}`

Query Parameters:

- `userId` (required): User ID
- `startDate` (optional): Filter start date
- `endDate` (optional): Filter end date

Response: *(blocco di codice rimosso)*

Permission System

PlayRecordPermissionChecker

Authorization logic for play sessions.

View Permissions

Users can view a session if:

1. They are the session creator
2. Session is `Group` visibility AND user is in the group
3. User is a player in the session

(blocco di codice rimosso)

Edit Permissions

Only the session creator can edit sessions.

(blocco di codice rimosso)

Domain Events

PlayRecordCreatedEvent

- **Trigger**: New session created
- **Consumers**: Analytics, notifications

PlayerAddedToSessionEvent

- **Trigger**: Player joins session
- **Consumers**: Session tracking, notifications

SessionScoreRecordedEvent

- **Trigger**: Score recorded for player
- **Consumers**: Leaderboard updates, statistics

PlayRecordStartedEvent

- **Trigger**: Session marked as in-progress
- **Consumers**: Activity tracking

PlayRecordCompletedEvent

- **Trigger**: Session completed

- **Consumers**: Statistics updates, achievements, notifications

PlayRecordUpdatedEvent

- **Trigger**: Session details modified
- **Consumers**: Audit log

Validation Rules

CreatePlayRecordCommand

- `GameName` is required (max 255 characters)
- `SessionDate` cannot be in the future
- If `GameId` provided, must exist in catalog
- If `Visibility` is `Group`, `GroupId` is required

AddPlayerToSessionCommand

- `DisplayName` is required (max 255 characters)
- Cannot add duplicate players (same UserId)
- Session must be in `Planned` or `InProgress` status

RecordSessionScoreCommand

- `PlayerId` must exist in session
- `Dimension` must be in session's `ScoringConfig.EnabledDimensions`
- `Value` must be non-negative for most dimensions

CompletePlayRecordCommand

- Session must not already be `Completed`
- If `ManualDuration` not provided, `StartTime` must be set

Error Responses

Status Code	Error	Description
400	`ValidationError`	Invalid request data
403	`ForbiddenError`	Insufficient permissions
404	`NotFoundError`	Session/Game/Player not found
409	`ConflictError`	Duplicate player, invalid state transition

Example Error Response: *(blocco di codice rimosso)*

Usage Examples

Creating a Session with Catalog Game

(blocco di codice rimosso)

Creating a Free-Form Session

(blocco di codice rimosso)

Recording a Complete Game Flow

(blocco di codice rimosso)

Related Documentation

- GameManagement Bounded Context
- Permission System
- CQRS Pattern
- Domain Events

Implementation Status

- [ ] Domain Model (#3875)
- [ ] CQRS Commands (#3876)
- [ ] CQRS Queries (#3877)
- [ ] Permission System (#3878)
- [ ] Frontend UI (#3879)

Parent Epic: #3874 **Target Release**: TBD

api/bounded-contexts/game-management/play-sessions.md

Play Sessions API Documentation

Overview

The Play Sessions feature enables users to record and track their board game play sessions with flexible player management, multi-dimensional scoring, and comprehensive statistics.

Bounded Context: GameManagement **Parent Epic**: #3874 **Implementation Status**: Planned

---

Domain Model

PlaySession (Aggregate Root)

The core entity representing a single game play session.

Key Properties

Property	Type	Description
`Id`	`Guid`	Unique session identifier
`GameId`	`Guid?`	Optional reference to Game in catalog
`GameName`	`string`	Game name (from catalog or user input)
`CreatedByUserId`	`Guid`	Session creator
`Visibility`	`PlaySessionVisibility`	Private or Group visibility
`GroupId`	`Guid?`	Optional group for shared sessions
`SessionDate`	`DateTime`	Date of play session
`StartTime`	`DateTime?`	Optional session start time
`EndTime`	`DateTime?`	Optional session end time
`Duration`	`TimeSpan?`	Calculated or manual duration
`Status`	`PlaySessionStatus`	Planned, InProgress, Completed, Archived
`Notes`	`string?`	Optional session notes
`Location`	`string?`	Optional play location
`Players`	`List<SessionPlayer>`	Participants in session
`ScoringConfig`	`SessionScoringConfig`	Scoring configuration

Factory Methods

(blocco di codice rimosso)

Behaviors

(blocco di codice rimosso)

SessionPlayer (Entity)

Represents a participant in a play session (registered user or guest).

Property	Type	Description
`Id`	`Guid`	Player identifier within session
`UserId`	`Guid?`	Optional User reference (null for guests)
`DisplayName`	`string`	Player display name
`Scores`	`List<SessionScore>`	Multi-dimensional scores

SessionScore (Value Object)

Multi-dimensional score representation.

Property	Type	Description
`Dimension`	`string`	Score dimension (e.g., "points", "ranking", "wins")
`Value`	`int`	Score value
`Unit`	`string?`	Optional display unit (e.g., "pts", "1st", "W")

Factory Methods: *(blocco di codice rimosso)*

SessionScoringConfig (Value Object)

Configuration for scoring dimensions available in the session.

Property	Type	Description
`EnabledDimensions`	`List<string>`	Active scoring dimensions
`DimensionUnits`	`Dictionary<string, string>`	Display units per dimension

Factory Methods: *(blocco di codice rimosso)*

Enumerations

PlaySessionStatus

- `Planned` - Session scheduled but not started
- `InProgress` - Session currently active

- `Completed` - Session finished
- `Archived` - Completed session archived

PlaySessionVisibility

- `Private` - Only creator can view
- `Group` - Visible to group members

Database Schema

PlaySessions Table

(blocco di codice rimosso)

Indexes: *(blocco di codice rimosso)*

SessionPlayers Table

(blocco di codice rimosso)

SessionScores Table

(blocco di codice rimosso)

CQRS Operations

Commands

CreatePlaySessionCommand

Creates a new play session.

Request: *(blocco di codice rimosso)*

Response: `Guid` (Session ID)

Endpoint: `POST /api/v1/game-management/play-sessions`

AddPlayerToSessionCommand

Adds a player to an existing session.

Request: *(blocco di codice rimosso)*

Endpoint: `POST /api/v1/game-management/play-sessions/{sessionId}/players`

RecordSessionScoreCommand

Records a score for a player in a session.

Request: *(blocco di codice rimosso)*

Endpoint: `POST /api/v1/game-management/play-sessions/{sessionId}/scores`

StartPlaySessionCommand

Marks a session as in-progress.

Endpoint: `POST /api/v1/game-management/play-sessions/{sessionId}/start`

CompletePlaySessionCommand

Completes a session and calculates duration.

Request: *(blocco di codice rimosso)*

Endpoint: `POST /api/v1/game-management/play-sessions/{sessionId}/complete`

UpdatePlaySessionCommand

Updates session details (allowed post-completion).

Request: *(blocco di codice rimosso)*

Endpoint: `PUT /api/v1/game-management/play-sessions/{sessionId}`

Queries

GetPlaySessionQuery

Retrieves full session details including players and scores.

Request: `GET /api/v1/game-management/play-sessions/{sessionId}`

Response: *(blocco di codice rimosso)*

GetUserPlayHistoryQuery

Retrieves paginated play history for a user.

Request: `GET /api/v1/game-management/play-sessions/history?userId={userId}&page=1&pageSize=20&gameId={gameId}`

Query Parameters:

- `userId` (required): User ID
- `page` (optional, default: 1): Page number
- `pageSize` (optional, default: 20): Results per page
- `gameId` (optional): Filter by specific game

Response: *(blocco di codice rimosso)*

GetPlayerStatisticsQuery

Retrieves cross-game statistics for a player.

Request: `GET /api/v1/game-management/play-sessions/statistics?userId={userId}&startDate={startDate}&endDate={endDate}`

Query Parameters:

- `userId` (required): User ID
- `startDate` (optional): Filter start date
- `endDate` (optional): Filter end date

Response: *(blocco di codice rimosso)*

Permission System

PlaySessionPermissionChecker

Authorization logic for play sessions.

View Permissions

Users can view a session if:

1. They are the session creator
2. Session is `Group` visibility AND user is in the group
3. User is a player in the session

(blocco di codice rimosso)

Edit Permissions

Only the session creator can edit sessions.

(blocco di codice rimosso)

Domain Events

PlaySessionCreatedEvent

- **Trigger**: New session created
- **Consumers**: Analytics, notifications

PlayerAddedToSessionEvent

- **Trigger**: Player joins session
- **Consumers**: Session tracking, notifications

SessionScoreRecordedEvent

- **Trigger**: Score recorded for player
- **Consumers**: Leaderboard updates, statistics

PlaySessionStartedEvent

- **Trigger**: Session marked as in-progress
- **Consumers**: Activity tracking

PlaySessionCompletedEvent

- **Trigger**: Session completed

- **Consumers**: Statistics updates, achievements, notifications

PlaySessionUpdatedEvent

- **Trigger**: Session details modified
- **Consumers**: Audit log

Validation Rules

CreatePlaySessionCommand

- `GameName` is required (max 255 characters)
- `SessionDate` cannot be in the future
- If `GameId` provided, must exist in catalog
- If `Visibility` is `Group`, `GroupId` is required

AddPlayerToSessionCommand

- `DisplayName` is required (max 255 characters)
- Cannot add duplicate players (same UserId)
- Session must be in `Planned` or `InProgress` status

RecordSessionScoreCommand

- `PlayerId` must exist in session
- `Dimension` must be in session's `ScoringConfig.EnabledDimensions`
- `Value` must be non-negative for most dimensions

CompletePlaySessionCommand

- Session must not already be `Completed`
- If `ManualDuration` not provided, `StartTime` must be set

Error Responses

Status Code	Error	Description
400	`ValidationError`	Invalid request data
403	`ForbiddenError`	Insufficient permissions
404	`NotFoundError`	Session/Game/Player not found
409	`ConflictError`	Duplicate player, invalid state transition

Example Error Response: *(blocco di codice rimosso)*

Usage Examples

Creating a Session with Catalog Game

(blocco di codice rimosso)

Creating a Free-Form Session

(blocco di codice rimosso)

Recording a Complete Game Flow

(blocco di codice rimosso)

Related Documentation

- GameManagement Bounded Context
- Permission System
- CQRS Pattern
- Domain Events

Implementation Status

- [ ] Domain Model (#3875)
- [ ] CQRS Commands (#3876)
- [ ] CQRS Queries (#3877)
- [ ] Permission System (#3878)
- [ ] Frontend UI (#3879)

Parent Epic: #3874 **Target Release**: TBD

api/endpoints/library-game-detail.md

Library Game Detail Endpoint

Issue: #3511 - Game Detail Page **Bounded Context**: UserLibrary **Last Updated**: 2026-02-06

Endpoint

(blocco di codice rimosso)

Retrieves comprehensive details for a game in the user's library.

Authentication

Required: Yes (User session via cookie)

Parameters

Path Parameters

Parameter	Type	Required	Description
`id`	`Guid`	Yes	The game ID from user's library

Response

Success (200 OK)

Returns `LibraryGameDetailDto`:

(blocco di codice rimosso)

Error Responses

401 Unauthorized

(blocco di codice rimosso)

404 Not Found

(blocco di codice rimosso)

500 Internal Server Error

(blocco di codice rimosso)

Example Request

(blocco di codice rimosso)

Example Response

(blocco di codice rimosso)

Implementation

Handler: `GetLibraryGameDetailQueryHandler.cs` **Query**: `GetLibraryGameDetailQuery.cs` **Route**: `UserLibraryEndpoints.cs`

Related Endpoints

- `PUT /api/v1/library/games/{id}/state` - Update game state
- `POST /api/v1/library/games/{id}/favorite` - Toggle favorite
- `PUT /api/v1/library/games/{id}/notes` - Update notes
- `POST /api/v1/library/games/{id}/labels` - Add label
- `DELETE /api/v1/library/games/{id}/labels/{labelId}` - Remove label
- `DELETE /api/v1/library/games/{id}` - Remove from library

Frontend Integration

Page: `apps/web/src/app/(authenticated)/library/games/[gameId]/page.tsx` **Hook**: `useLibraryGameDetail(gameId)` from `@/hooks/queries/useLibrary`

Usage Example

(blocco di codice rimosso)

Notes

- All timestamps returned in ISO 8601 UTC format
- Rating values are nullable (not all games have ratings)
- PDF documents and social links arrays may be empty
- Play statistics (timesPlayed, winRate, etc.) are user-specific aggregations
- The endpoint joins data from SharedGameCatalog (public) and UserLibraryEntry (user-specific)

Testing

Unit Tests: `GetLibraryGameDetailQueryHandlerTests.cs` **Integration Tests**: Test with real database via Testcontainers **E2E Tests**: `apps/web/__tests__/e2e/library/game-detail.spec.ts`

api/health-check-api.md

Health Check API Reference

> ***Last Updated***: *2026-01-17* ***Related***: *Health Check System* ***Issue***: *#2511*

Endpoints

Endpoint	Purpose	Auth
`/api/v1/health`	Comprehensive health check (all services)	None
`/health/ready`	Readiness probe (critical services only, Kubernetes)	None
`/health/live`	Liveness probe (application process, Kubernetes)	None

`/api/v1/health` - Comprehensive Check

Request

(blocco di codice rimosso)

Response

Status: `200 OK` (always) or `503` (health system failure)

Schema: *(blocco di codice rimosso)*

Fields:

Field	Type	Values	Description
overallStatus	string	Healthy, Degraded, Unhealthy	System health summary
checks[]	array	-	Individual service checks
serviceName	string	postgres, redis, qdrant, etc.	Service identifier
status	string	Healthy, Degraded, Unhealthy	Service status
description	string	-	Human-readable message
isCritical	boolean	true/false	Core functionality dependency
timestamp	string	ISO 8601	Check execution time

Overall Status Logic:

Status	Condition
Healthy	All checks return Healthy
Degraded	≥1 non-critical service is Degraded/Unhealthy
Unhealthy	≥1 critical service is Unhealthy

Service Inventory

Service	Criticality	Check Type	Timeout	Endpoint/Method
postgres	🔴 CRITICAL	Database	5s	`SELECT 1`
redis	🔴 CRITICAL	Cache	5s	`PING`
qdrant	🔴 CRITICAL	HTTP	5s	`GET /healthz`
embedding	🔴 CRITICAL	HTTP	5s	`GET /health`
openrouter	🟡 IMPORTANT	HTTP	5s	`GET /api/v1/models`
bgg-api	🟡 IMPORTANT	HTTP	10s	`GET /xmlapi2/thing?id=1`
unstructured	🟡 IMPORTANT	HTTP	5s	`GET /health`
reranker	🟢 OPTIONAL	HTTP	5s	`GET /health`
smoldocling	🟢 OPTIONAL	HTTP	5s	`GET /health`
email-smtp	🟢 OPTIONAL	Config	0s	Configuration check
oauth-*	🟢 OPTIONAL	Config	0s	Configuration check
grafana	🟢 OPTIONAL	HTTP	5s	`GET /api/health`
prometheus	🟢 OPTIONAL	HTTP	5s	`GET /-/healthy`
hyperdx	🟢 OPTIONAL	Config	0s	Configuration check

Response Examples

All Services Healthy

(blocco di codice rimosso)

Degraded (Non-Critical Down)

(blocco di codice rimosso)

Interpretation: Application functional with reduced features (no OpenRouter, BGG sync paused)

Unhealthy (Critical Down)

(blocco di codice rimosso)

Interpretation: Core functionality broken (database unavailable) - immediate action required

---

`/health/ready` - Readiness Probe

Purpose: Kubernetes readiness - can accept traffic?

Request: `GET http://localhost:8080/health/ready`

Response:

- `200 OK` + `"Healthy"` → Ready
- `503 Service Unavailable` + `"Unhealthy"` → Not ready

Checks: Critical services only (postgres, redis, qdrant, embedding)

Kubernetes Config: *(blocco di codice rimosso)*

`/health/live` - Liveness Probe

Purpose: Kubernetes liveness - is process running?

Request: `GET http://localhost:8080/health/live`

Response:

- `200 OK` + `"Healthy"` → Alive
- No response → Crashed (Kubernetes restarts)

Checks: Application process only (no external dependencies)

Kubernetes Config: *(blocco di codice rimosso)*

Integration Examples

Prometheus Monitoring

Scrape Config (`prometheus.yml`): *(blocco di codice rimosso)*

Alert Rules: *(blocco di codice rimosso)*

Load Balancer (Nginx)

(blocco di codice rimosso)

CI/CD Validation

(blocco di codice rimosso)

Troubleshooting

Common Issues

Issue	Diagnosis	Resolution
All Unhealthy	App not started or network isolation	Check `docker compose ps api` and logs
Timeout	External service not responding	Identify slow service with `curl -w` timing
Degraded (all secrets OK)	External API rate limiting	Check specific service in response, wait for rate limit reset

Error Codes

Code	Meaning	Action
200 OK	Check completed	Review `overallStatus` in response body
503 Service Unavailable	Health system failed	Restart app, check logs
404 Not Found	Wrong endpoint	Use `/api/v1/health` not `/health`

Testing

Unit Tests: `tests/Api.Tests/Infrastructure/Health/PostgresHealthCheckTests.cs` (per-service) **Integration Tests**: `tests/Api.Tests/Routing/HealthEndpointsTests.cs` (endpoint validation) **E2E Tests** (Future): `apps/web/e2e/admin/health-dashboard.spec.ts` (admin UI)

Example Test: *(blocco di codice rimosso)*

Related Documentation

- Health Check System Overview
- Auto-Configuration Guide
- Deployment Guide

Maintained by: MeepleAI API Team **Questions**: GitHub Issues

---

api/multi-agent-system.md

Multi-Agent AI System - Complete Guide

Generated: 2026-02-12 | **Epic**: #3490 | **Issue**: #3780

Overview

MeepleAI Multi-Agent System provides specialized AI assistance for board games through three intelligent agents:

1. **Tutor Agent**: Interactive tutorials, setup guidance, rules Q&A
2. **Arbitro Agent**: Real-time move validation, rules arbitration
3. **Decisore Agent**: Strategic move suggestions, position analysis

Foundation: Context Engineering Framework (RAG evolution), LangGraph orchestration, hybrid search

---

Architecture

(blocco di codice rimosso)

API Reference

Decisore Agent

Endpoint: `POST /api/v1/agents/decisore/analyze`

Authentication: Requires active session token

Request: *(blocco di codice rimosso)*

Response (200 OK): *(blocco di codice rimosso)*

Performance:

- Quick: <1s (heuristics only)
- Standard: <3s (heuristics + LLM top 3)
- Deep: <5s (multi-model ensemble)

Errors:

- 400: Invalid request (bad depth, maxSuggestions out of range)
- 401: Unauthorized (no session)
- 404: GameSession not found
- 500: Analysis timeout

Arbitro Agent

Endpoint: `POST /api/v1/agents/arbitro/validate`

Request: *(blocco di codice rimosso)*

Response: *(blocco di codice rimosso)*

Performance: P95 <100ms (with Redis tier-1 cache)

Tutor Agent

Endpoint: `POST /api/v1/agents/tutor/query`

Request: *(blocco di codice rimosso)*

Response: *(blocco di codice rimosso)*

User Guide

Using Decisore Agent

1. Start a game session: *(blocco di codice rimosso)*

2. Request strategic analysis: *(blocco di codice rimosso)*

3. Review suggestions:

- Each suggestion includes move notation, reasoning, pros/cons
- Position strength shows overall evaluation
- Victory paths provide strategic direction

Admin Guide

Configuring Agents

AgentDefinition Management:

Create Agent Template: *(blocco di codice rimosso)*

Available Types:

- RAG: Retrieval-Augmented Generation
- Citation: Source validation
- Confidence: Quality assessment
- RulesInterpreter: Game rules specialist
- Conversation: Chat management
- Custom: User-defined

Available Strategies:

- HybridSearch: Vector (70%) + Keyword (30%)
- VectorOnly: Pure semantic search
- MultiModelConsensus: Multi-LLM agreement
- CitationValidation: Source verification
- ConfidenceScoring: Multi-layer quality check
- Custom: User-defined parameters

Troubleshooting

Common Issues

1. Decisore returns no suggestions

- **Cause**: No legal moves (stalemate/checkmate)
- **Solution**: Check game state, verify position is valid

2. Analysis timeout (>5s)

- **Cause**: Complex position with many candidates
- **Solution**: Use "quick" depth or reduce maxSuggestions

3. Low confidence scores

- **Cause**: Unclear position or move quality similar

- **Solution**: Normal in balanced positions, review multiple suggestions

4. Arbitro "UNCERTAIN" decision

- **Cause**: Rule conflicts or missing rules
- **Solution**: Check rule coverage, add FAQ resolution

Performance Benchmarks

Decisore Agent (Issue #3774)

Depth	Target	Typical	Components
Quick	<1s	800ms	Heuristics only
Standard	<3s	2.5s	Heuristics + LLM top 3
Deep	<5s	4.2s	Multi-model ensemble

Component Breakdown (Standard):

- Parse FEN: 45ms
- Generate 30 moves: 380ms
- Validate (parallel): 95ms
- Score heuristics: 180ms
- LLM top-3 (parallel): 850ms
- Format response: 25ms
- **Total**: ~1575ms ✅

Arbitro Agent (Issue #3874)

Metric	Target	Actual
P50 Latency	<50ms	35ms
P95 Latency	<100ms	68ms
P99 Latency	<200ms	142ms
Cache Hit Rate	>80%	87%

Examples

Example 1: Get Strategic Advice

Request: *(blocco di codice rimosso)*

Response: *(blocco di codice rimosso)*

Technical Implementation

Move Generation Pipeline (Issue #3770)

ChessMoveGenerator:

1. Get player pieces
2. Generate pseudo-legal moves (6 piece types)
3. Validate legality (check detection)
4. Score with heuristics (material + positional + tactical)
5. Rank by priority
6. Return top N

Performance: <500ms for 30-40 legal moves

Strategic Analysis (Issue #3769)

DecisoreAgentService:

1. Parse game state (#3772)
2. Generate candidates (#3770)
3. Refine top 3 with LLM
4. Evaluate position strength
5. Identify victory paths
6. Calculate risk level

Performance: <3s standard, <5s deep

Multi-Model Ensemble (Issue #3771)

MultiModelEvaluator:

1. Parallel LLM calls (GPT-4, Claude, DeepSeek)

2. Calculate consensus (mean score)
3. Detect disagreement (variance)
4. Adjust confidence (high agreement = 95%)

Performance: ~900ms (parallel) vs 2400ms (sequential)

Related Documentation

- Context Engineering Framework
- Decisore Implementation Plan
- Move Generator Spec
- Agent Builder Plan

Last Updated: 2026-02-12 **Contributors**: Claude Sonnet 4.5 **Status**: Complete - All 3 agents documented

---

api/permission-api-reference.md

Permission API Reference (Epic #4068)

Base URL: `/api/v1/permissions` **Authentication**: Required (Bearer token) **Rate Limit**: 100 requests/minute per user

---

Endpoints

GET /me

Description: Get current user's permission context and accessible features

Authentication: Required

Request: *(blocco di codice rimosso)*

Response 200 OK: *(blocco di codice rimosso)*

Response Fields:

- `tier` (string): User's subscription tier
  - Values: `"free"`, `"normal"`, `"premium"`, `"pro"`, `"enterprise"`
- `role` (string): User's role
  - Values: `"user"`, `"editor"`, `"creator"`, `"admin"`, `"superadmin"`
- `status` (string): Account status
  - Values: `"Active"`, `"Suspended"`, `"Banned"`
- `limits` (object): Tier-based collection limits
  - `maxGames` (number): Max games in collection
  - `storageQuotaMB` (number): Max storage quota in MB
- `accessibleFeatures` (string[]): List of features user can access

Error Responses:

`401 Unauthorized`: *(blocco di codice rimosso)*

`404 Not Found`: *(blocco di codice rimosso)*

Caching: Frontend should cache for 5 minutes (staleTime: 5min)

Example Usage:

(blocco di codice rimosso)

(blocco di codice rimosso)

GET /check

Description: Check if user has access to specific feature (optionally with resource state)

Authentication: Required

Request: *(blocco di codice rimosso)*

Query Parameters:

- `feature` (required, string): Feature name to check
  - Examples: `"wishlist"`, `"bulk-select"`, `"quick-action.delete"`
- `state` (optional, string): Resource state for state-based checks
  - Examples: `"draft"`, `"published"`, `"archived"` (for games)

Response 200 OK: *(blocco di codice rimosso)*

Response Fields:

- `hasAccess` (boolean): True if user has access, false otherwise
- `reason` (string): Explanation for result
  - Values: `"Tier sufficient"`, `"Role sufficient"`, `"Neither tier nor role sufficient"`, `"User account is suspended"`, etc.
- `details` (object): Detailed permission context
  - `userTier` (string): User's tier display name
  - `userRole` (string): User's role display name
  - `userStatus` (string): Account status
  - `required` (object): Requirements for this feature
    - `tier` (string | null): Required tier (if any)
    - `role` (string | null): Required role (if any)
    - `states` (string[] | null): Allowed states (if any)
  - `logic` (string): Permission logic (`"Or"` or `"And"`)

Examples:

Example 1: Pro user checking bulk-select (allowed) *(blocco di codice rimosso)*

Response: *(blocco di codice rimosso)*

Example 2: Free user checking bulk-select (denied) *(blocco di codice rimosso)*

Response: *(blocco di codice rimosso)*

Example 3: State-based check (draft game access) *(blocco di codice rimosso)*

Response (creator): *(blocco di codice rimosso)*

Response (regular user): *(blocco di codice rimosso)*

Error Responses:

`400 Bad Request` (missing feature parameter): *(blocco di codice rimosso)*

`404 Not Found` (unknown feature): *(blocco di codice rimosso)*

Feature Permission Registry

Registered Features (as of Epic #4068):

Feature Name	Required Tier	Required Role	Logic	Description
`wishlist`	Free	User	OR	Add games to wishlist
`bulk-select`	Pro	Editor	OR	Select multiple cards
`drag-drop`	Normal	User	OR	Drag to reorder
`quick-action.delete`	Free	Admin	AND	Delete via quick action menu
`quick-action.edit`	Normal	Creator	OR	Edit via quick action menu
`collection.manage`	Normal	User	OR	Create/edit collections
`document.upload`	Normal	User	OR	Upload PDF rulebooks
`agent.create`	Pro	Creator	OR	Create AI agents
`analytics.view`	Pro	Admin	OR	View analytics dashboard
`filters.advanced`	Normal	User	OR	Use advanced filters

Adding New Features:

(blocco di codice rimosso)

Testing New Features: *(blocco di codice rimosso)*

Client Libraries

TypeScript Client

(blocco di codice rimosso)

React Hooks

(blocco di codice rimosso)

C# Client (Server-to-Server)

(blocco di codice rimosso)

Rate Limiting

Limits:

- 100 requests/minute per user (across both endpoints)
- 1000 requests/minute globally

Response when rate limited (429 Too Many Requests): *(blocco di codice rimosso)*

Client Handling: *(blocco di codice rimosso)*

Webhooks (Future Enhancement)

Permission Change Events (not yet implemented, planned for v1.6):

(blocco di codice rimosso)

Use Cases:

- Invalidate permission caches in frontend
- Trigger welcome email for tier upgrade
- Update analytics dashboards
- Notify connected WebSocket clients

Monitoring & Observability

Metrics Endpoint

GET /metrics (Prometheus format): *(blocco di codice rimosso)*

Logging

Info Level: Successful permission checks *(blocco di codice rimosso)*

Warning Level: Denied permission checks *(blocco di codice rimosso)*

Error Level: Failures (should be rare) *(blocco di codice rimosso)*

Versioning

Current: v1 (Epic #4068)

Future Versions:

- v1.1: Add batch permission check endpoint (`POST /check/batch`)
- v1.2: Add permission cache endpoint (`GET /cache/{userId}`)
- v2.0: Breaking changes (remove deprecated fields)

Deprecation Policy:

- Deprecated fields marked in response (with `_deprecated: true`)
- Deprecated endpoints: 6-month notice before removal
- Version in URL (future): `/api/v2/permissions/me`

Security Considerations

Token Validation:

- Bearer token signature verified
- Token expiration checked (15min for access tokens)
- User ID claim extracted from token

Authorization:

- All endpoints require authentication (`.RequireAuthorization()`)

- Tier/role read from database (not JWT claims alone)
- Permission checks logged for security monitoring

Input Validation:

- Feature name: Whitelist validation against PermissionRegistry
- State: Optional, validated if provided
- User ID: GUID format validation

Output Sanitization:

- Generic error messages (don't leak internal details)
- Sensitive info logged server-side only (not returned in response)

Client Best Practices

1. Cache Aggressively *(blocco di codice rimosso)*

2. Batch Checks (when v1.1 available) *(blocco di codice rimosso)*

3. Handle Errors Gracefully *(blocco di codice rimosso)*

4. Optimistic UI Updates *(blocco di codice rimosso)*

Integration Examples

Example 1: Feature-Locked Button

(blocco di codice rimosso)

Example 2: Admin Panel Guard

(blocco di codice rimosso)

Example 3: Dynamic Menu Items

(blocco di codice rimosso)

Testing the API

Manual Testing with curl

(blocco di codice rimosso)

Automated Integration Tests

(blocco di codice rimosso)

Performance Considerations

Endpoint Latency:

- `/me`: ~5-20ms (single SELECT query + registry lookup)
- `/check`: ~2-10ms (cached in PermissionRegistry singleton)

Optimization:

- PermissionRegistry: Singleton (initialized once)
- Database query: Indexed SELECT on Tier/Role/Status
- Response caching: Consider HybridCache for /me (future)

Scaling:

- 10K concurrent users: ~100 req/sec to /me (peak)
- Database: Single SELECT per request (lightweight)
- No N+1 queries (projection used)

API Changelog

v1.0 (2026-02-12) - Initial release (Epic #4068)

- `GET /api/v1/permissions/me`: Get user permissions
- `GET /api/v1/permissions/check`: Check feature access
- PermissionRegistry: 10 initial features

Planned v1.1 (2026-03-15):

- `POST /api/v1/permissions/check/batch`: Batch permission checks
- `GET /api/v1/permissions/features`: List all registered features

- Webhook events for permission changes

Planned v2.0 (2026-06-01):

- Breaking: Remove deprecated IsSuspended field
- Breaking: Require state parameter for state-based features
- New: Resource-level permission checks (`/check/{resourceType}/{resourceId}`)

References

- Epic #4068: https://github.com/DegrassiAaron/meepleai-monorepo/issues/4068
- Issue #4177: Permission Data Model & Schema
- ADR-050: Permission System Architecture
- OpenAPI Spec: http://localhost:8080/scalar/v1 (after `dotnet run`)

---

api/session-tracking/sse-integration.md

GST-003: Server-Sent Events (SSE) Integration Guide

Overview

Real-time session synchronization using Server-Sent Events (SSE) for multi-device collaborative game sessions.

Why SSE over WebSockets:

- ✅ Simpler one-way server→client communication
- ✅ Automatic browser reconnection
- ✅ HTTP/2 compatible (works with Traefik reverse proxy)
- ✅ Stateless backend (no persistent connection state)

Architecture

Components

Service Layer:

- `ISessionSyncService` - Domain interface for event subscription/publishing
- `SessionSyncService` - In-memory Channel-based pub/sub implementation

Event Types (6 total):

- `ScoreUpdatedEvent` - Real-time score changes
- `ParticipantAddedEvent` - New player joins
- `NoteAddedEvent` - Shared notes broadcast (Private notes not included)
- `SessionFinalizedEvent` - Session closed with final rankings
- `SessionPausedEvent` - Placeholder for future pause flow
- `SessionResumedEvent` - Placeholder for future resume flow

Integration:

- Command handlers publish events after `SaveChangesAsync()`
- SSE endpoint streams events to authenticated clients
- Heartbeat every 30s for connection keep-alive

API Endpoint

SSE Stream

(blocco di codice rimosso)

Response Headers: *(blocco di codice rimosso)*

Event Format: *(blocco di codice rimosso)*

Authorization:

- User must be session owner OR participant
- Returns 401 if unauthenticated
- Returns 403 if not authorized for session
- Returns 404 if session not found

Frontend Integration

Basic EventSource Setup

(blocco di codice rimosso)

React Hook Example

(blocco di codice rimosso)

Event Schemas

ScoreUpdatedEvent

(blocco di codice rimosso)

ParticipantAddedEvent

(blocco di codice rimosso)

NoteAddedEvent

(blocco di codice rimosso)

SessionFinalizedEvent

(blocco di codice rimosso)

Error Handling

Client-Side Errors

Connection Failure (401/403/404): *(blocco di codice rimosso)*

Reconnection Strategy:

- Browser auto-reconnects with `Last-Event-ID` header
- Exponential backoff: 1s → 2s → 4s → 8s → max 30s
- Reset attempts counter on successful connection

Server-Side Errors

CancellationToken Propagation:

- Client disconnect → CancellationToken triggered
- Channel automatically cleaned up
- Memory freed immediately

Concurrent Access:

- Thread-safe `ConcurrentDictionary` for subscribers
- Channel writes are non-blocking
- No deadlocks under load

Performance Characteristics

Latency

- **Event broadcast:** <100ms from command completion to SSE delivery
- **Heartbeat timing:** 30±5s intervals
- **Connection overhead:** ~2KB/min (heartbeat only)

Scalability

- **Current:** In-memory implementation (single-instance)
- **Subscribers per session:** Unlimited (tested up to 1000)
- **Memory per subscriber:** ~8KB (channel + buffer)
- **Future:** Redis pub/sub backend for multi-instance (GST-003 Phase 2)

Resource Management

- Auto-cleanup on disconnect (no manual intervention)
- No memory leaks (channels disposed properly)
- Graceful shutdown support (CancellationToken)

Testing

Unit Tests

(blocco di codice rimosso)

Coverage:

- Service layer: 90%+
- Handler integration: 95%+
- Event types: 85%+

Manual Testing with curl

Subscribe to events: *(blocco di codice rimosso)*

Browser DevTools Testing

(blocco di codice rimosso)

Deployment Considerations

Reverse Proxy (Traefik)

Buffering must be disabled: *(blocco di codice rimosso)*

Timeouts: *(blocco di codice rimosso)*

Load Balancer

Sticky sessions required for in-memory:

- Client must connect to same backend instance
- Use session ID in cookie for routing
- Alternative: Redis backend for multi-instance

Health checks:

- SSE endpoint excluded from load balancer health checks
- Use `/health` endpoint for health monitoring

Future Enhancements (Phase 2)

Redis Pub/Sub Backend

(blocco di codice rimosso)

Benefits:

- Multi-instance deployment support
- Event persistence for replay capability
- No sticky session requirement
- Better scalability

Trade-offs:

- Additional latency (~200ms vs <100ms)

- Redis dependency
- Network overhead

Event Replay

(blocco di codice rimosso)

Use case:

- Client reconnects after disconnect
- Replay missed events since `lastEventId`
- Prevents state desynchronization

Troubleshooting

Issue: Events not received

Check:

1. Authorization: Verify session cookie or Bearer token
2. Network: Check browser Network tab for 200 response
3. Buffering: Ensure Traefik/nginx buffering disabled
4. CORS: Verify `withCredentials: true` if cross-origin

Debug: *(blocco di codice rimosso)*

Issue: Connection drops frequently

Check:

1. Heartbeat: Verify 30s heartbeat in Network tab
2. Timeout: Check reverse proxy read timeout settings
3. Network: Mobile networks may close idle connections

Solution:

- Reduce heartbeat interval to 15s if needed
- Implement client-side ping/pong

Issue: Memory leak on server

Verify: *(blocco di codice rimosso)*

Check:

- CancellationToken properly propagated
- Channels disposed in `finally` block
- No exceptions preventing cleanup

Related Issues

- **GST-001**: SessionTracking Foundation
- **GST-002**: CQRS Commands & Queries
- **GST-003**: Real-Time SSE Infrastructure (this document)
- **GST-004**: Frontend Real-Time UI Integration (requires this)

Last Updated: 2026-01-30 **Status:** Production Ready **Version:** 1.0